

Лабораторная работа № 3.3 «Семантический анализ»

1 июня 2026 г.

Ольга Ивенкова, ИУ9-62Б

Цель работы

Целью данной работы является получение навыков выполнения семантического анализа.

Индивидуальный вариант

Семантический анализ для варианта ЛР2.2

Подмножество Рефала-5

```
/* Декартово произведение */
CartProd {
  (t.X e.X) (e.Y) = <CartProd-Bind t.X e.Y> <CartProd (e.X) (e.Y)>;
  (/ * пусто */) (e.Y) = / * пусто */;
}

CartProd-Bind {
  t.X t.Y e.Y = (t.X t.Y) <CartProd-Bind t.X e.Y>;
  t.X / * пусто */ = / * пусто */;
}

/*
  Прибавление 1 к строковой записи числа:
  123 → 124, 99 → 100, 007 → 008
*/
Inc {
  e.Prefix s.Last, '0123456789' : e.1 s.Last s.Next e.2 = e.Prefix s.Next;
  e.Prefix '9' = <Inc e.Prefix> '0';
  / * пусто */ = '1';
}
```

```

/* Функция возвращает уникальные идентификаторы с заданным префиксом */
NextId {
  e.prefix
  , <Dg counter>
  : {
    s.n = e.prefix <Symb s.n> <Br counter '=' <Add 1 s.n>>;
    /* пусто */ = <Br counter '=' 0> <NextId e.prefix>;
  };
}

```

Семантический анализ

Проверки:

- В программе не может быть двух одноимённых функций.
- Вызываемая функция должна быть либо определена в программе, либо быть встроенной (в примере выше это Br и Dg, полный список см. в руководстве по Рефалу-5).
- К переменным можно обращаться, если они определены ранее в предложении (например, в образце условия).

Результат:

Прогонка Рефала в общем случае сложна, т.к. требует сопоставления параметризованного выражения с образцом общего вида. А это приводит к решению уравнений в словах и прочим сложностям.

На практике можно выбрать достаточно выразительное подмножество — подмножество т.н. ограниченного Рефала, в котором образцы являются т.н. L-образцами.

L-образцы не могут содержать повторных t- и e-переменных и не могут содержать более одной e-переменной на одном скобочном уровне. Следующие образцы **не являются** L-образцами:

- t.X e.Y t.X — повторная t-переменная,
- t.X (e.Y A e.Z) — две e-переменные на одном уровне,
- e.1 (e.2) e.3 — две e-переменные на одном уровне.

А эти принадлежат: (e.1) e.2 (e.3), s.X e.Y s.X, t.X (e.Y A) e.Z.

Задача состоит в том, чтобы распечатать список функций, принадлежащих подмножеству ограниченного Рефала. Функция принадлежит подмножеству, если все образцы в ней являются образцами ограниченного Рефала (включая образцы в условиях и блоках).

Заметим, что переменная может быть повторной в двух случаях:

- Переменная непосредственно встречается в образце более одного раза.
- Одна из переменных образца ранее встречалась в этом же предложении.

В следующем примере переменная `e.X` в образце условия — повторная, а значит функция `As` не является функцией ограниченного Рефала:

```
As {
  e.X, A e.X : e.X A = True;
  e.Y = False;
}
```

TL;DR Чтобы проверить, что функция является функцией ограниченного Рефала, нужно перебрать в ней все образцы и проверить для каждого из них:

- на верхнем уровне образца и в каждом скобочном терме не более одной `e`-переменной,
- в образцах `e`- и `t`-переменные не встречаются дважды,
- `e`- и `t`-переменные, которые встречаются в этом образце, не встречались в предшествующих образцах предложения.

Реализация

```
from __future__ import annotations

import ast
import sys
from dataclasses import dataclass, field
from abc import ABC, abstractmethod
from typing import Any, Optional, Set, List, Dict
import parser_edsl as pe

class SemanticError(pe.Error):
    pass

class DuplicateFunction(SemanticError):
    def __init__(self, pos, name):
        self.pos = pos
        self.name = name
    @property
    def message(self):
        return f"Повторное определение функции {self.name}"

class UnknownFunction(SemanticError):
    def __init__(self, pos, name):
        self.pos = pos
        self.name = name
    @property
    def message(self):
        return f"Неизвестная функция {self.name}"
```

```

class UndefinedVariable(SemanticError):
    def __init__(self, pos, name):
        self.pos = pos
        self.name = name
    @property
    def message(self):
        return f"Необъявленная переменная {self.name}"

# -----
class Symbol:
    def __init__(self, name: str, info: Any):
        self.name = name
        self.info = info

class SymbolTable:
    def __init__(self, outer: Optional['SymbolTable'] = None):
        self.outer = outer
        self.symbols: Dict[str, Symbol] = {}
        self.open_scopes: List[SymbolTable] = []

    def add(self, sym: Symbol):
        self.symbols[sym.name] = sym

    def contains(self, name: str) -> bool:
        return name in self.symbols

    def contains_visible(self, name: str) -> bool:
        return self.find(name) is not None

    def find(self, name: str) -> Optional[Symbol]:
        if name in self.symbols:
            return self.symbols[name]
        if self.outer:
            return self.outer.find(name)
        for scope in self.open_scopes:
            sym = scope.find(name)
            if sym:
                return sym
        return None

    def merge(self, other: 'SymbolTable'):
        for name, sym in other.symbols.items():
            if name not in self.symbols:
                self.symbols[name] = sym

```

```

# Program -> Function*
@dataclass
class Program:
    functions: List[FunctionDef]
    def check(self, builtins: Set[str]) -> List[str]:
        func_table = {}
        for f in self.functions:
            if f.name in func_table:
                raise DuplicateFunction(f.name_coord, f.name)
            func_table[f.name] = f
        good_functions = []
        for f in self.functions:
            f.check(func_table, builtins)
            if f.is_restricted():
                good_functions.append(f.name)
        return good_functions

# Function -> IDENT { Sentence* }
@dataclass
class FunctionDef:
    name: str
    name_coord: pe.Position
    body: List[Sentence]
    _restricted: Optional[bool] = field(default=None, repr=False)

    def check(self, func_table: Dict[str, FunctionDef], builtins: Set[str]):
        for sent in self.body:
            sent.check(func_table, builtins, outer_symtab=None)
        self._restricted = all(s.is_restricted for s in self.body)

    def is_restricted(self) -> bool:
        return bool(self._restricted)

# Sentence -> LeftPart RightPart ;
@dataclass
class Sentence:
    left: LeftPart
    right: RightPart
    is_restricted: bool = field(default=False, repr=False)

    def check(self,
        func_table: Dict[str, FunctionDef],
        builtins: Set[str],
        outer_symtab: Optional[SymbolTable] = None):
        symtab = SymbolTable(outer=outer_symtab) if outer_symtab else SymbolTable()
        self.left.check(symtab, func_table, builtins)

```

```

        self.right.check(symtab, func_table, builtins)
        self.is_restricted = self.left.is_restricted

# LeftPart -> Pattern ( Condition* )
@dataclass
class LeftPart:
    pattern: PatternExpr
    conditions: List[Condition]
    is_restricted: bool = field(default=False, repr=False)

    def check(self,
              symtab: SymbolTable,
              func_table: Dict[str, FunctionDef],
              builtins: Set[str]):
        pattern_ok = self._check_pattern(self.pattern, symtab)
        cond_ok = True
        for cond in self.conditions:
            cond_symtab = SymbolTable(outer=symtab)
            cond_ok = cond_ok and self._check_pattern(cond.pattern, cond_symtab)
            symtab.merge(cond_symtab)
        self.is_restricted = pattern_ok and cond_ok

    def _check_pattern(self, pat: PatternExpr, symtab: SymbolTable) -> bool:
        checker = PatternChecker(symtab)
        pat.accept(checker)
        return checker.is_restricted()

class PatternChecker:
    def __init__(self, symtab: SymbolTable):
        self.symtab = symtab
        self.restricted = True
        self.evar_count = 0
        self.seen_vars = set()
        self.group_level = 0

    def is_restricted(self) -> bool:
        return self.restricted

    def visit_literal(self, lit):
        pass

    def visit_var(self, var: VarRef, fullname: str):
        if var.kind in ('t', 'e'):
            # Повтор внутри текущего образца
            if fullname in self.seen_vars:
                self.restricted = False

```

```

        # Повтор с переменной, объявленной ранее в предложении
        if self.symtab.contains_visible(fullname):
            self.restricted = False
            self.seen_vars.add(fullname)

    self.symtab.add(Symbol(fullname, var))

    # e-переменные не более одной на уровне
    if var.kind == 'e':
        self.evar_count += 1
        if self.evar_count > 1:
            self.restricted = False

    def enter_group(self):
        self.group_level += 1
        self.evar_count = 0

    def exit_group(self):
        self.group_level -= 1

# Condition -> , Result : Pattern
@dataclass
class Condition:
    result: ResultExpr
    pattern: PatternExpr

class RightPart(ABC):
    @abstractmethod
    def check(self,
              symtab: SymbolTable,
              func_table: Dict[str, FunctionDef],
              builtins: Set[str]):
        pass

# RightPart -> = Result
@dataclass
class ReturnRightPart(RightPart):
    expr: ResultExpr
    def check(self, symtab, func_table, builtins):
        self.expr.check(symtab, func_table, builtins)

# RightPart -> , Result : { Sentences }
@dataclass
class BlockRightPart(RightPart):
    expr: ResultExpr
    body: List[Sentence]

```

```

    def check(self, symtab, func_table, builtins):
        self.expr.check(symtab, func_table, builtins)
        for sent in self.body:
            sent.check(func_table, builtins, outer_symtab=symtab)

# -----
class ResultItem(ABC):
    @abstractmethod
    def check(self,
               symtab: SymbolTable,
               func_table: Dict[str, FunctionDef],
               builtins: Set[str]):
        pass

class PatternItem(ABC):
    @abstractmethod
    def accept(self, visitor: PatternChecker):
        pass

# Result -> ResultItem*
@dataclass
class ResultExpr:
    items: List[ResultItem]
    def check(self, symtab, func_table, builtins):
        for item in self.items:
            item.check(symtab, func_table, builtins)

# Pattern -> PatternItem*
@dataclass
class PatternExpr:
    items: List[PatternItem]
    def accept(self, visitor: PatternChecker):
        for item in self.items:
            item.accept(visitor)

# STRING
@dataclass
class StringLiteral(ResultItem, PatternItem):
    value: str
    def check(self, symtab, func_table, builtins):
        pass
    def accept(self, visitor):
        visitor.visit_literal(self)

# INTEGER
@dataclass

```

```

class IntLiteral(ResultItem, PatternItem):
    value: int
    def check(self, symtab, func_table, builtins):
        pass
    def accept(self, visitor):
        visitor.visit_literal(self)

# IDENT
@dataclass
class IdentLiteral(ResultItem, PatternItem):
    name: str
    def check(self, symtab, func_table, builtins):
        pass
    def accept(self, visitor):
        visitor.visit_literal(self)

# VAR
@dataclass
class VarRef(ResultItem, PatternItem):
    kind: str
    index: str | int
    coord: pe.Position = field(default=None)
    def check(self, symtab, func_table, builtins):
        name = f"{self.kind}.{self.index}"
        if symtab.find(name) is None:
            raise UndefinedVariable(self.coord, name)
    def accept(self, visitor):
        visitor.visit_var(self, f"{self.kind}.{self.index}")

# (Result)
@dataclass
class ResultGroup(ResultItem):
    expr: ResultExpr
    def check(self, symtab, func_table, builtins):
        self.expr.check(symtab, func_table, builtins)

# (Pattern)
@dataclass
class PatternGroup(PatternItem):
    expr: PatternExpr
    def accept(self, visitor):
        visitor.enter_group()
        self.expr.accept(visitor)
        visitor.exit_group()

# <IDENT Result>

```

```

@dataclass
class CallExpr(ResultItem):
    name: str
    args: ResultExpr
    coord: pe.Position = field(default=None)
    def check(self, symtab, func_table, builtins):
        if self.name not in func_table and self.name not in builtins:
            raise UnknownFunction(self.coord, self.name)
        self.args.check(symtab, func_table, builtins)

# -----

def parse_string_literal(image: str) -> str:
    return ast.literal_eval(image)

def parse_var_ref(image: str) -> VarRef:
    kind, index_text = image.split('.', 1)
    index: str | int = int(index_text) if index_text.isdigit() else index_text
    return VarRef(kind=kind, index=index)

STRING = pe.Terminal('STRING', r'"(?:\\.|[^\\"])*"', parse_string_literal)
INTEGER = pe.Terminal('INTEGER', r'[0-9]+', int)
IDENT = pe.Terminal('IDENT', r'[A-Za-z][A-Za-z0-9_-]*', str)
VAR = pe.Terminal('VAR', r'(?s|t|e)\.(?:[A-Za-z][A-Za-z0-9_-]*|[0-9]+)(?![A-Za-z0-9_-])',
    parse_var_ref)

NProgram = pe.NonTerminal('Program')
NFunctions = pe.NonTerminal('Functions')
NFunction = pe.NonTerminal('Function')
NSentences = pe.NonTerminal('Sentences')
NSentence = pe.NonTerminal('Sentence')
NLeftPart = pe.NonTerminal('LeftPart')
NConditions = pe.NonTerminal('Conditions')
NCondition = pe.NonTerminal('Condition')
NRightPart = pe.NonTerminal('RightPart')
NPatternExpr = pe.NonTerminal('PatternExpr')
NPatternItems = pe.NonTerminal('PatternItems')
NPatternItem = pe.NonTerminal('PatternItem')
NResultExpr = pe.NonTerminal('ResultExpr')
NResultItems = pe.NonTerminal('ResultItems')
NResultItem = pe.NonTerminal('ResultItem')

@pe.ExAction
def make_program(attrs, coords, res_coord):
    return Program(attrs[0])

```

```

@pe.ExAction
def make_function(attrs, coords, res_coord):
    name, sentences = attrs
    return FunctionDef(name, coords[0].start, sentences)

@pe.ExAction
def make_sentence(attrs, coords, res_coord):
    left, right = attrs
    return Sentence(left, right)

@pe.ExAction
def make_left_part(attrs, coords, res_coord):
    pat, conds = attrs
    return LeftPart(pat, conds)

@pe.ExAction
def make_condition(attrs, coords, res_coord):
    result, pattern = attrs
    return Condition(result, pattern)

@pe.ExAction
def make_return_right(attrs, coords, res_coord):
    return ReturnRightPart(attrs[0])

@pe.ExAction
def make_block_right(attrs, coords, res_coord):
    expr, sentences = attrs
    return BlockRightPart(expr, sentences)

@pe.ExAction
def make_pattern_expr(attrs, coords, res_coord):
    return PatternExpr(attrs[0])

@pe.ExAction
def make_pattern_group(attrs, coords, res_coord):
    return PatternGroup(attrs[0])

@pe.ExAction
def make_result_expr(attrs, coords, res_coord):
    return ResultExpr(attrs[0])

@pe.ExAction
def make_result_group(attrs, coords, res_coord):
    return ResultGroup(attrs[0])

@pe.ExAction

```

```

def make_call(attrs, coords, res_coord):
    name, args = attrs
    return CallExpr(name, args, coords[1].start)

@pe.ExAction
def make_var_ref(attrs, coords, res_coord):
    var = attrs[0]
    var.coord = coords[0].start
    return var

@pe.ExAction
def make_string_literal(attrs, coords, res_coord):
    return StringLiteral(attrs[0])

@pe.ExAction
def make_int_literal(attrs, coords, res_coord):
    return IntLiteral(attrs[0])

@pe.ExAction
def make_ident_literal(attrs, coords, res_coord):
    return IdentLiteral(attrs[0])

NProgram |= NFunctions, make_program
NFunctions |= lambda: []
NFunctions |= NFunction, NFunctions, lambda f, rest: [f] + rest

NFunction |= IDENT, '{', NSentences, '}', make_function
NSentences |= lambda: []
NSentences |= NSentence, NSentences, lambda s, rest: [s] + rest

NSentence |= NLeftPart, NRightPart, ';', make_sentence
NLeftPart |= NPatternExpr, NConditions, make_left_part
NConditions |= lambda: []
NConditions |= NCondition, NConditions, lambda c, rest: [c] + rest
NCondition |= ',', NResultExpr, ':', NPatternExpr, make_condition

NRightPart |= '=', NResultExpr, make_return_right
NRightPart |= ',', NResultExpr, ':', '{', NSentences, '}', make_block_right

NPatternExpr |= NPatternItems, make_pattern_expr
NPatternItems |= lambda: []
NPatternItems |= NPatternItem, NPatternItems, lambda item, rest: [item] + rest

NPatternItem |= STRING, make_string_literal
NPatternItem |= INTEGER, make_int_literal
NPatternItem |= IDENT, make_ident_literal

```

```

NPatternItem |= VAR, make_var_ref
NPatternItem |= '(', NPatternExpr, ')', make_pattern_group

NResultExpr |= NResultItems, make_result_expr
NResultItems |= lambda: []
NResultItems |= NResultItem, NResultItems, lambda item, rest: [item] + rest

NResultItem |= STRING, make_string_literal
NResultItem |= INTEGER, make_int_literal
NResultItem |= IDENT, make_ident_literal
NResultItem |= VAR, make_var_ref
NResultItem |= '(', NResultExpr, ')', make_result_group
NResultItem |= '<', IDENT, NResultExpr, '>', make_call

def build_parser() -> pe.Parser:
    parser = pe.Parser(NProgram, method=pe.EARLEY)
    parser.add_skipped_domain(r'\s+')
    parser.add_skipped_domain(r'/*[\s\S]*?\/')
    return parser

BUILTINS = {'Br', 'Dg', 'Add', 'Symb'}

def main():
    parser = build_parser()
    for filename in sys.argv[1:]:
        try:
            with open(filename, 'r', encoding='utf-8') as f:
                source = f.read()
                tree = parser.parse(source)
                good = tree.check(BUILTINS)
                print("Программа корректна")
                if good:
                    print("Функции, принадлежащие подмножеству ограниченного Рефала:")
                    for name in good:
                        print(f"\t{name}")
                else:
                    print("Нет функций, принадлежащих подмножеству ограниченного Рефала")
        except pe.Error as e:
            print(f"Ошибка {e.pos}: {e.message}")
        except Exception as e:
            print(f"Необработанная ошибка: {e}")

if __name__ == '__main__':
    main()

```

Тестирование

Входные данные

```
CartProd-Bind {
  t.X t.Y e.Y = (t.X t.Y) <CartProd-Bind t.X e.Y>;
  t.X /* пусто */ = /* пусто */;
}

/*
  Прибавление 1 к строковой записи числа:
  123 → 124, 99 → 100, 007 → 008
*/
Inc {
  e.Prefix s.Last, '0123456789' : e.1 s.Last s.Next e.2 = e.Prefix s.Next;
  e.Prefix '9' = <Inc e.Prefix> '0';
  /* пусто */ = '1';
}

/* Функция возвращает уникальные идентификаторы с заданным префиксом */
NextId {
  e.prefix
  , <Dg counter>
  : {
    s.n = e.prefix <Symb s.n> <Br counter '=' <Add 1 s.n>>;
    /* пусто */ = <Br counter '=' 0> <NextId e.prefix>;
  };
}

As {
  e.X, A e.X : e.X A = True;
  e.Y = False;
}

eee {
  e.1 (e.2) e.3 = 1;
}
```

Вывод на stdout

Программа корректна
Функции, принадлежащие подмножеству ограниченного Рефала:
CartProd-Bind
NextId

Вывод

Получила навыки выполнения семантического анализа.